

TIẾT 2: Chuỗi ký tự, Ký tự thoát & Kỹ năng chẩn đoán, gỡ lỗi

Lập trình Java - Lớp IT26M

Thời lượng: 50 phút | 35’ Lý thuyết & Thảo luận + 15’ Hands-on Lab 2

Gemini Notebook



Lý thuyết:

Chuỗi ký tự (String Literal) là một chuỗi các ký tự nằm trong cặp dấu ngoặc kép (“...”).
Ví dụ: “Xin chao”, “IT26M”, “” (chuỗi rỗng).



Không được ngắt một chuỗi qua nhiều dòng mà không đóng ngoặc kép!

Ngữ cảnh: Nếu muốn in ra màn hình chính xác câu: Giao vien noi: “Java rat thu vi!”

Warning

main.java

```
1
2  System.out.println("Giao vien
     noi: "Java rat thu vi!");
3
```

Giải thích lỗi: Trình biên dịch sẽ báo lỗi ngay! Vì nó nghĩ chuỗi kết thúc ở dấu ngoặc kép thứ hai.

Gemini Notebook

Ký tự thoát chuẩn (Escape Sequences) - Quy tắc: Dấu gạch chéo ngược (\) kết hợp với ký tự đi liền sau.

	**\t**	**Tab**	Thêm khoảng thụt lề tab (căn chỉnh cột dữ liệu đẹp mắt).
	**\n**	**Newline**	Xuống dòng mới ngay tại vị trí xuất hiện.
	**\"**	**Double quote**	In dấu ngoặc kép " mà không làm đóng chuỗi.
	****	**Backslash**	In một dấu gạch chéo ngược ****.

Listing 3

main.java

```java
System.out.println("Cot 1\tCot 2\tCot 3");
System.out.println("Dong 1\nDong 2\nDong 3");
System.out.println("Dau \"nhay kep\" va gach \");
```

```
Cot 1    Cot 2    Cot 3
Dong 1
Dong 2
Dong 3
Dau "nhay kep" va gach \
```

Gemini Notebook

Định danh (Identifiers) – Tên do lập trình viên đặt cho lớp, phương thức, biến...

Tiêu chuẩn chung: Ký tự hợp lệ: Các chữ cái (**a-z, A-Z**), chữ số (**0-9**), dấu gạch dưới (**_**), dấu đô la (**$**).

Quy tắc 1: Không được bắt đầu bằng chữ số!

✅ diem1

❌ 1diem

Quy tắc 2: Không chứa dấu cách hoặc ký tự đặc biệt (@, #, %, -, +).

Quy tắc 3: Không trùng với Từ khóa Java!

Gemini Notebook

Từ khóa chuẩn Java (Keywords – Dành riêng)

Java có 50+ từ khóa đặc biệt đã được định nghĩa sẵn cú pháp.



Grid Box

`public`	`class`	`static`	`void`
`int`	`double`	`boolean`	`char`
`if`	`else`	`for`	`while`
`return`	`new`	`true`	`false`

Tuyệt đối không dùng các từ trên làm tên Class hay tên hàm!

Gemini Notebook



Phong cách lập trình chuẩn đại học

Tên Lớp: UpperCamelCase: Viết hoa chữ cái đầu của mỗi từ.

Hello **D**raw**F**igures3 **S**tudent**M**anager

Tên Phương thức & Biến: **lowerCamelCase**: Chữ cái đầu viết thường, viết hoa các từ tiếp theo.

main draw**D**iamond calculate**T**otal**S**core

Thụt lề: Indentation: Nhất quán 4 khoảng trắng!

Gemini Notebook

3 Loại chú thích (Comments) trong Java

Chú thích 1 dòng

Cú pháp: `//`

```
// Tinh tong diem
cua sinh vien
```

Chú thích nhiều dòng

Cú pháp: `/* ... */`

```
/* Tac gia: IT26M
   Ngay tao: 2026 */
```

Chú thích tài liệu (Javadoc)

Cú pháp: `/** ... */`

Dùng để sinh tài liệu API tự động.

“Mã nguồn viết ra cho người đọc hiểu trước, máy tính hiểu sau.”

Gemini Notebook



Phân loại lỗi lập trình: 3 nhóm lỗi kinh điển

Mọi lập trình viên từ sinh viên năm nhất đến chuyên gia thâm niên đều phải đối mặt với lỗi!

Cú pháp - Syntax Errors

Thời điểm: Biên dịch (Compile-time). Trình biên dịch javac từ chối sinh file .class.

Đặc điểm: Vi phạm ngữ pháp (thiếu dấu ;, sai từ khóa, quên đóng }).

Thực thi - Runtime Errors

Thời điểm: Đang chạy (Run-time). Chương trình bị sập (crash) và tung ra ngoại lệ (Exception).

Đặc điểm: CPU thực hiện phép toán bất khả thi (chia cho 0, truy cập mảng rỗng).

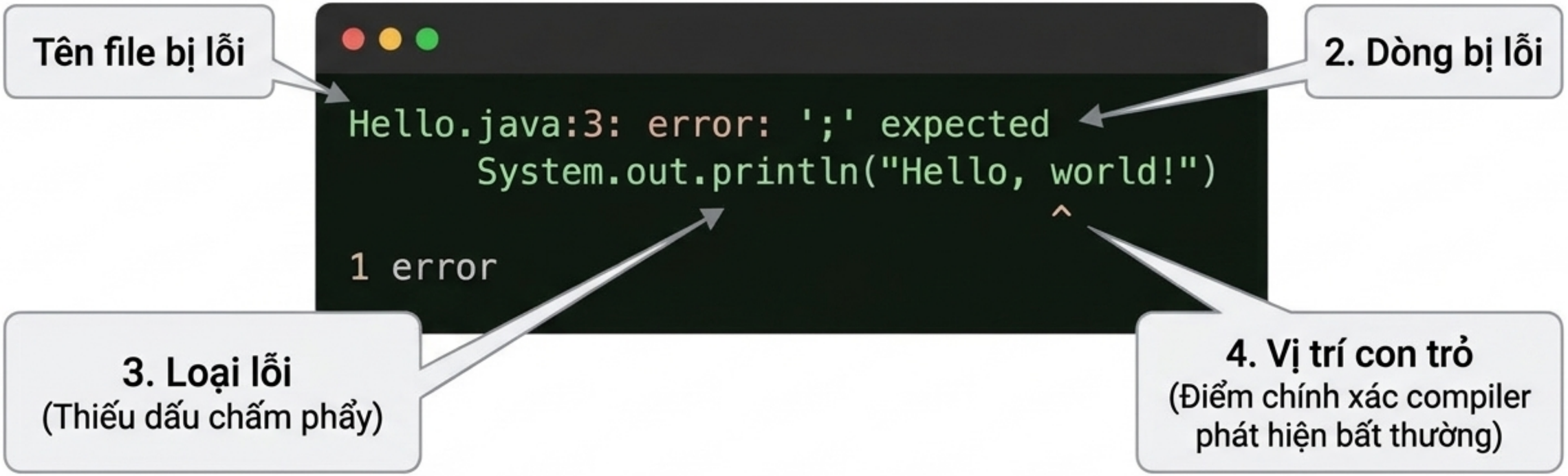
Ngữ nghĩa - Logic Errors

Thời điểm: Chạy không sập, nhưng kết quả sai.

Đặc điểm: Nguy hiểm nhất vì máy tính không thể tự phát hiện! (VD: Cần tính tiền thuế mà viết nhầm dấu + thành dấu -).

Gemini Notebook

Kỹ năng đọc thông báo lỗi từ Trình biên dịch



Kinh nghiệm vàng khi Debug

- **Quy tắc 1:** Luôn nhìn vào lỗi đầu tiên! Một lỗi ở dòng 3 có thể tạo 10 lỗi giả phía sau.
- **Quy tắc 2:** Nếu dòng báo lỗi trông hoàn toàn đúng, hãy kiểm tra dòng ngay phía trước nó.

Gemini Notebook

[CHECKPOINT 2] Thử tài thám tử chẩn đoán lỗi



```
1    public MyProgram {
2        public static void Main(String[] args) {
3            System.out.println("Lop IT26M);
4        }
5    }
```

Chẩn đoán và phân tích chi tiết:

1. **Lỗi dòng 1:** Thiếu từ khóa class → Sửa: **public class** MyProgram.
2. **Lỗi dòng 2:** Tên hàm main bị viết hoa chữ M (Java phân biệt hoa thường) → Sửa: **main**.
3. **Lỗi dòng 3:** Chuỗi ký tự thiếu ngoặc kép đóng " và dấu chấm phẩy ; → Sửa: System.out.println(**“Lop IT26M”**);.

Gemini Notebook



[HANDS-ON LAB 2] Thực hành Debug & Chuỗi ký tự thoát

Bài toán: Viết chương trình EscapeChallenge.java (15 phút).

Gợi ý kỹ thuật:

- Đường dẫn: Dùng \\.
- Khẩu hiệu: Dùng ".
- Thụt lề: Dùng \t.

Thử thách: Cố tình gõ thiếu dấu \ để xem trình biên dịch báo lỗi gì!

```
Dac ta he thong:
    1. Thu muc:  C:\Java\IT26M
    2. Khau hieu: "Hoc la phai hanh!"
    3. Tam giac:  /\ /\
```

Gemini Notebook



TIẾT 3: Phân rã thủ tục, Phương thức tĩnh & Tiến hóa Case Study

Tư duy Kiến trúc (Procedural Decomposition)

Thời lượng: 50 phút | 30’ Lý thuyết thiết kế + 20’ Mini Project Lab 3

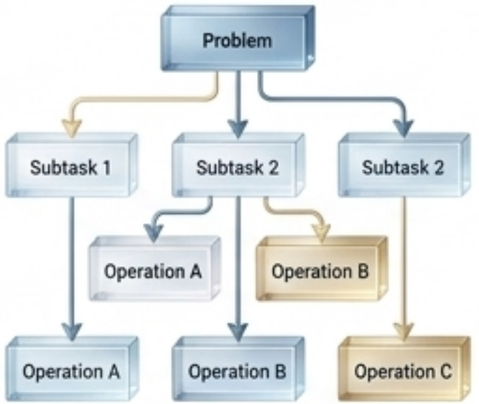
Gemini Notebook

Cạm bẫy của chương trình nguyên khối - Monolithic



- Tất cả hàng trăm dòng lệnh trong hàm main.
- Mã nguồn cực kỳ khó đọc (spaghetti code).
- Trùng lặp mã nguồn (Code Duplication): Gõ đi gõ lại, khó sửa lỗi.

Phân rã Top-down - Divide and Conquer



- Chia bài toán lớn thành các bài toán con (subtasks).
- Tiếp tục chia nhỏ đến các thao tác đơn giản nhất.
- Ví dụ: Sơ đồ “Make Cake” (Chia thành Làm bột, Nướng, Làm kem).
- Mỗi hộp trong sơ đồ = Một phương thức tĩnh (static method) trong Java.

Nguyên lý cốt lõi: DRY – Don’t Repeat Yourself!

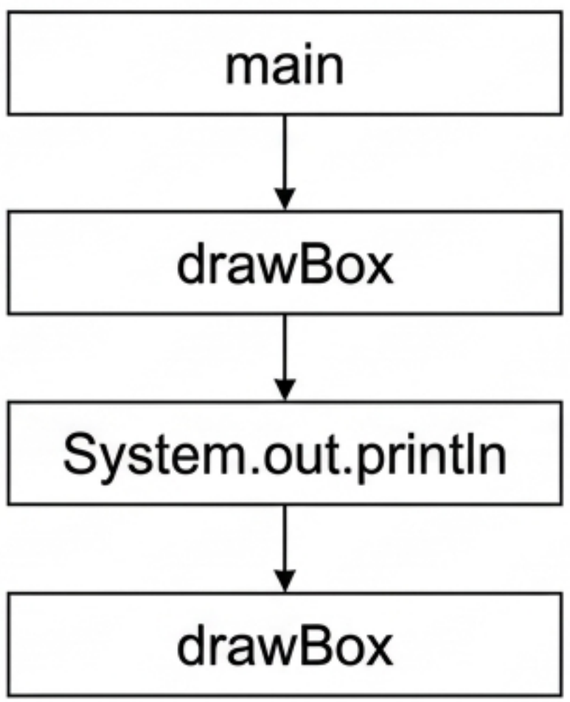
Gemini Notebook

Cú pháp **Khai báo** - Định nghĩa hành vi:

```java
public static void drawBox() {
    // Các câu lệnh in hình hộp
}
```

Cú pháp **Lời gọi** - Ra lệnh thực thi:

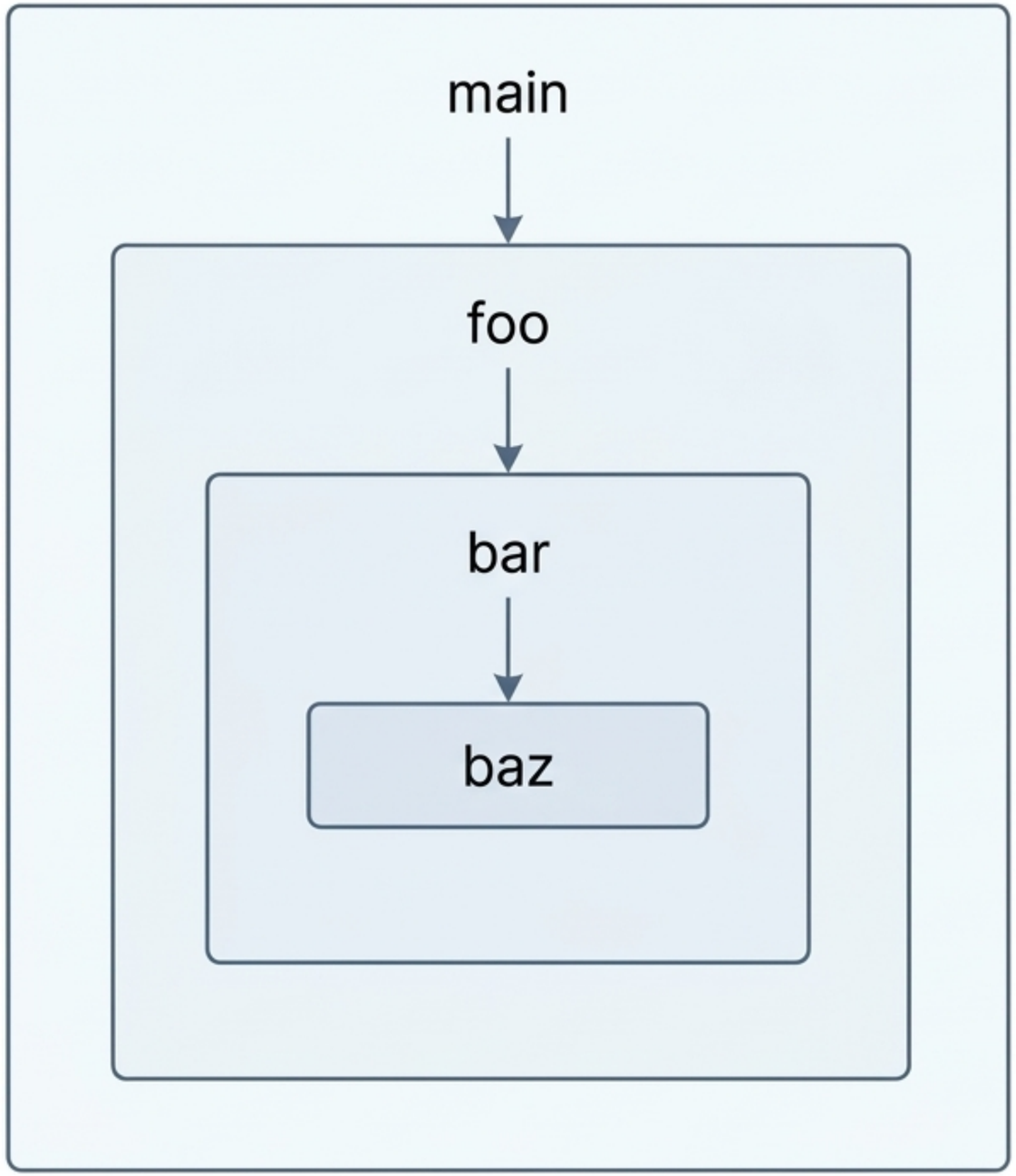
```java
public static void main(String[] args) {
    drawBox(); // Gọi lần 1
    System.out.println();
    drawBox(); // Gọi lần 2 (Tái sử dụng!)
}
```



Quy tắc chuyển giao:
Tạm dừng tại main → Nhảy tới **drawBox** thực thi → Quay về (**return**) đúng vị trí cũ để tiếp tục.



Gemini Notebook



Ngăn xếp gọi hàm (Call Stack)



- **Một phương thức có thể gọi phương thức khác.**
- Máy tính ghi nhớ thứ tự quay về bằng Call Stack **(LIFO: Vào sau ra trước).**
- Ví dụ: baz() chạy xong → quay về bar() → bar() chạy xong → quay về main.

Lợi ích kép của Phân rã thủ tục

1. **Structure** (Cấu trúc): Biến chương trình phức tạp thành module dễ hiểu.
2. **Eliminate Redundancy** (Xóa bỏ trùng lặp): Viết 1 lần, gọi thực thi vô số lần!

Gemini Notebook